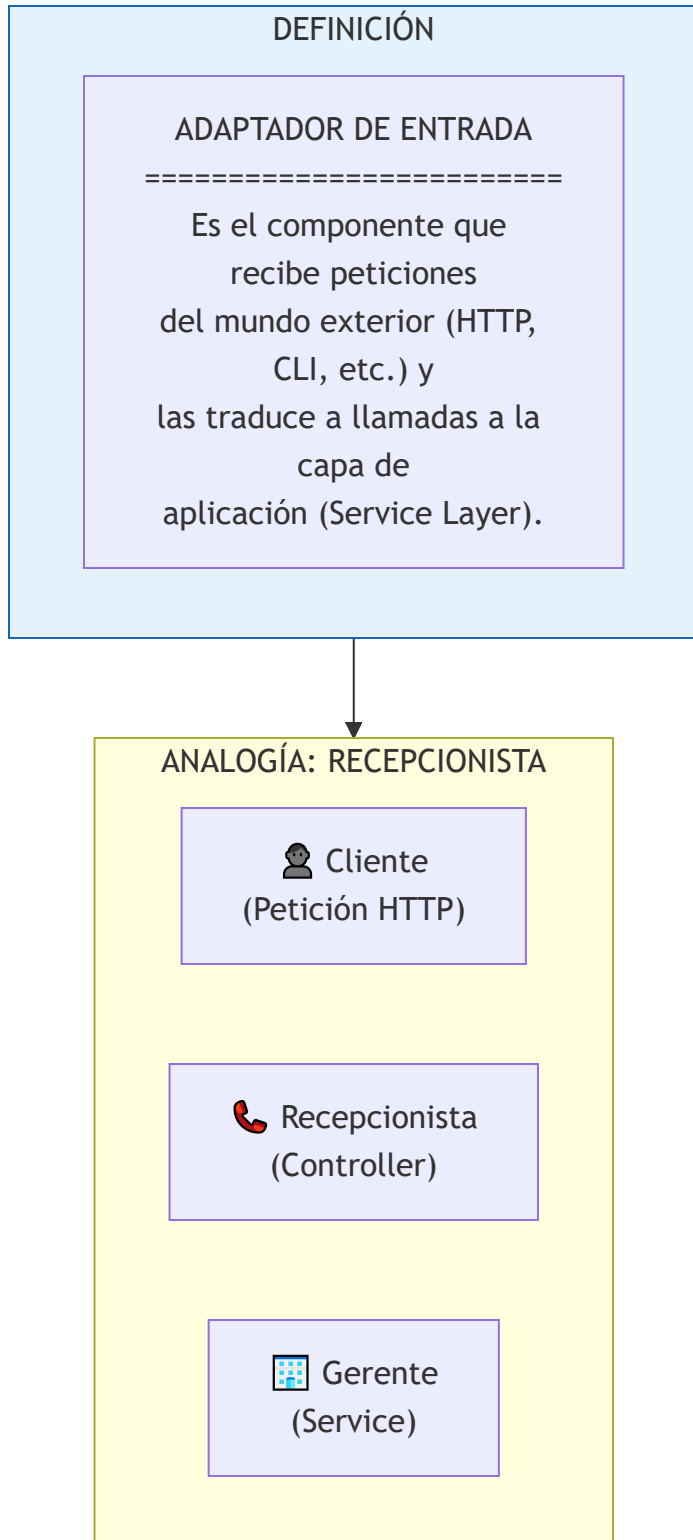


Guía 5: Adaptadores de Entrada - Controller y Routes

Índice

1. [¿Qué es un Adaptador de Entrada?](#)
2. [Arquitectura y Flujo de Datos](#)
3. [Implementación del EmployeeController](#)
4. [Implementación de Routes](#)
5. [Integración con Express Server](#)
6. [Manejo de Errores](#)
7. [Pruebas de la API](#)
8. [Ejercicios de Profundización](#)
9. [Checklist de Completitud](#)

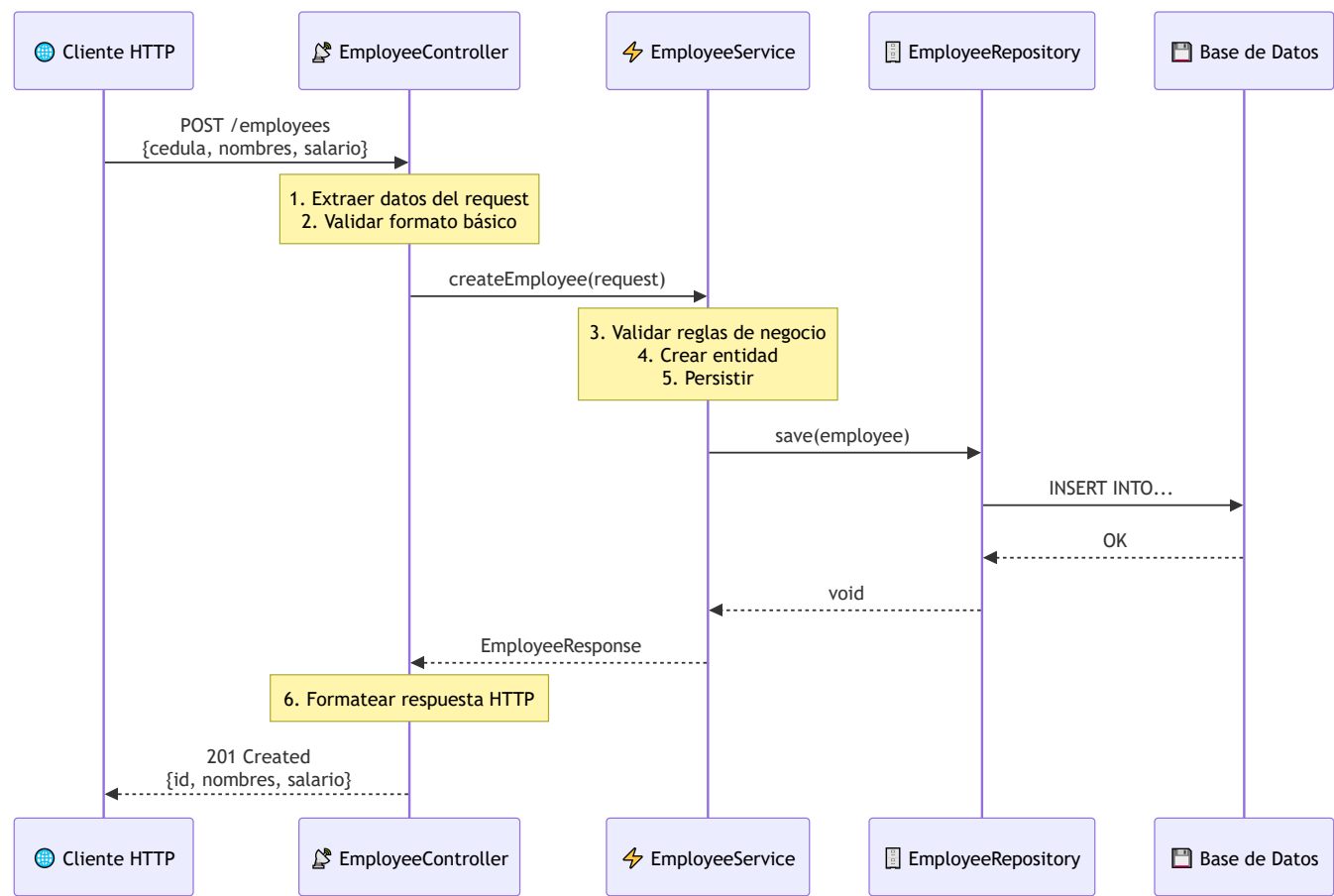
1. ¿Qué es un Adaptador de Entrada?



Rol del Controller en Hexagonal

Responsabilidad	¿Qué hace?	¿Qué NO hace?
Recibir peticiones	Escucha en endpoints HTTP	✗ No contiene lógica de negocio
Extraer datos	Obtiene datos del request (body, params, query)	✗ No valida reglas de dominio
Llamar al Service	Invoca métodos del Service Layer	✗ No accede directamente a repositorios
Formatear respuesta	Convierte resultados a JSON	✗ No formatea para otras capas
Manejar errores HTTP	Captura excepciones y retorna códigos HTTP	✗ No maneja errores de negocio

2. Arquitectura y Flujo de Datos



Ubicación en la Estructura del Proyecto

```
src/  
├─ domain/  
│   ├── employee.entity.ts  
│   └─ employee.repository.ts  
├─ application/  
│   ├── dtos/  
│   │   ├── requests/  
│   │   │   ├── create-employee.request.ts  
│   │   │   └─ update-employee.request.ts  
│   │   └─ responses/  
│   │       └─ employee.response.ts  
│   └─ index.ts  
├─ services/  
│   └─ employee.service.ts  
├─ infrastructure/  
│   ├── repositories/  
│   │   └─ employee.inmemory.repository.ts  
│   ├── controllers/          ← ¡CREAR ESTA CARPETA!  
│   │   └─ employee.controller.ts  
│   ├── routes/              ← ¡CREAR ESTA CARPETA!  
│   │   └─ employee.routes.ts  
│   └─ server.ts  
└─ app.ts
```

3. Implementación del EmployeeController

3.1 Crear la Carpeta y Archivo

```
# Crear la carpeta controllers  
mkdir -p src/infrastructure/controllers  
  
# Crear el archivo del controlador  
touch src/infrastructure/controllers/employee.controller.ts
```

3.2 Código del Controlador

```
// 📁 src/infrastructure/controllers/employee.controller.ts

import { Request, Response } from 'express';
import { EmployeeService } from '../../application/services/employee.service.js';
import { CreateEmployeeRequest, UpdateEmployeeRequest } from '../../application/dtos/index.js';

/**
 * EmployeeController
 * =====
 * Adaptador de entrada para el sistema de empleados.
 * Recibe peticiones HTTP y las traduce a llamadas al Service Layer.
 *
 * @example
 * ```typescript
 * const controller = new EmployeeController(employeeService);
 * app.post('/api/employees', controller.createEmployee);
 * ```
 */
export class EmployeeController {
  constructor(private readonly employeeService: EmployeeService) {}

  /**
   * POST /api/employees
   * Crea un nuevo empleado
   *
   * @param req - Petición HTTP con { cedula, nombres, salario }
   * @param res - Respuesta HTTP
   *
   * @returns 201 Created con el empleado creado
   * @returns 400 Bad Request si los datos son inválidos
   * @returns 409 Conflict si la cédula ya existe
   * @returns 500 Internal Server Error en caso de error
   */
  createEmployee = async (req: Request, res: Response): Promise<void> => {
    try {
      // 1. Extraer datos del request
      const { cedula, nombres, salario } = req.body;

      // 2. Validación básica (formato)
      if (!cedula || !nombres || salario === undefined) {
        res.status(400).json({

```

```

        error: 'Faltan campos requeridos: cedula, nombres, salario'
    });
    return;
}

// 3. Crear DTO de request
const request: CreateEmployeeRequest = {
    cedula,
    nombres,
    salario
};

// 4. Llamar al Service
const employee = await this.employeeService.createEmployee(request);

// 5. Responder con 201 Created
res.status(201).json({
    message: 'Empleado creado exitosamente',
    data: employee
});

} catch (error: any) {
    // 6. Manejo de errores específicos
    this.handleError(error, res);
}
};

/**
 * GET /api/employees/:id
 * Obtiene un empleado por su ID
 *
 * @param req - Petición HTTP con { id } en params
 * @param res - Respuesta HTTP
 */
getEmployeeById = async (req: Request, res: Response): Promise<void> => {
    try {
        const { id } = req.params;

        if (!id) {
            res.status(400).json({ error: 'ID es requerido' });
            return;
        }
    }

```

```

        const employee = await this.employeeService.getEmployeeById(id);

        res.status(200).json({
            data: employee
        });

    } catch (error: any) {
        this.handleError(error, res);
    }
};

/**
 * GET /api/employees
 * Lista todos los empleados
 *
 * @param req - Petición HTTP
 * @param res - Respuesta HTTP
 */
getAllEmployees = async (req: Request, res: Response): Promise<void> => {
    try {
        const employees = await this.employeeService.getAllEmployees();

        res.status(200).json({
            count: employees.length,
            data: employees
        });

    } catch (error: any) {
        this.handleError(error, res);
    }
};

/**
 * PUT /api/employees/:id
 * Actualiza un empleado existente
 *
 * @param req - Petición HTTP con { id } en params y { nombres, salario } en body
 * @param res - Respuesta HTTP
 */
updateEmployee = async (req: Request, res: Response): Promise<void> => {
    try {
        const { id } = req.params;
        const { nombres, salario } = req.body;

```



```

    if (!id) {
        res.status(400).json({ error: 'ID es requerido' });
        return;
    }

    // Verificar que hay datos para actualizar
    if (!nombres && salario === undefined) {
        res.status(400).json({
            error: 'Debe proporcionar al menos un campo para actualizar'
        });
        return;
    }

    const request: UpdateEmployeeRequest = {
        nombres,
        salario
    };

    const employee = await this.employeeService.updateEmployee(id, request);

    res.status(200).json({
        message: 'Empleado actualizado exitosamente',
        data: employee
    });

} catch (error: any) {
    this.handleError(error, res);
}

};

/**
 * DELETE /api/employees/:id
 * Elimina un empleado
 *
 * @param req - Petición HTTP con { id } en params
 * @param res - Respuesta HTTP
 */
deleteEmployee = async (req: Request, res: Response): Promise<void> => {
    try {
        const { id } = req.params;

        if (!id) {

```

```

        res.status(400).json({ error: 'ID es requerido' });
        return;
    }

    await this.employeeService.deleteEmployee(id);

    res.status(200).json({
        message: 'Empleado eliminado exitosamente'
    });

    } catch (error: any) {
        this.handleError(error, res);
    }
};

/**
 * PATCH /api/employees/:id/salary
 * Aumenta el salario de un empleado en un porcentaje
 *
 * @param req - Petición HTTP con { id } en params y { percentage } en body
 * @param res - Respuesta HTTP
 */
increaseSalary = async (req: Request, res: Response): Promise<void> => {
    try {
        const { id } = req.params;
        const { percentage } = req.body;

        if (!id) {
            res.status(400).json({ error: 'ID es requerido' });
            return;
        }

        if (!percentage || percentage < 0 || percentage > 100) {
            res.status(400).json({
                error: 'El porcentaje debe estar entre 0 y 100'
            });
            return;
        }

        const employee = await this.employeeService.increaseSalary(id, percentage);

        res.status(200).json({
            message: `Salario aumentado en ${percentage}%`,

```

```

        data: employee
    });

    } catch (error: any) {
        this.handleError(error, res);
    }
};

/**
 * GET /api/employees/stats
 * Obtiene estadísticas de los empleados
 *
 * @param req - Petición HTTP
 * @param res - Respuesta HTTP
 */
getStatistics = async (req: Request, res: Response): Promise<void> => {
    try {
        const stats = await this.employeeService.getStatistics();

        res.status(200).json({
            data: stats
        });

    } catch (error: any) {
        this.handleError(error, res);
    }
};

/**
 * GET /api/employees/salary-range
 * Busca empleados por rango de salario
 *
 * @param req - Petición HTTP con { min, max } en query
 * @param res - Respuesta HTTP
 */
getEmployeesBySalaryRange = async (req: Request, res: Response): Promise<void> => {
    try {
        const min = Number(req.query.min);
        const max = Number(req.query.max);

        if (isNaN(min) || isNaN(max)) {
            res.status(400).json({
                error: 'Los parámetros min y max deben ser números'
            });
        }
    }
};

```

```

    });
    return;
}

if (min < 0 || max < 0 || min > max) {
    res.status(400).json({
        error: 'Rango de salario inválido'
    });
    return;
}

const employees = await this.employeeService.getEmployeesBySalaryRange(min, max);

res.status(200).json({
    count: employees.length,
    data: employees
});

} catch (error: any) {
    this.handleError(error, res);
}
};

/**
 * Manejo centralizado de errores
 */
private handleError(error: any, res: Response): void {
    console.error('❌ Error en EmployeeController:', error.message);

    // Errores de validación (Bad Request)
    if (error.message.includes('cédula') ||
        error.message.includes('nombres') ||
        error.message.includes('salario') ||
        error.message.includes('porcentaje')) {
        res.status(400).json({ error: error.message });
        return;
    }

    // Error de duplicado (Conflict)
    if (error.message.includes('Ya existe')) {
        res.status(409).json({ error: error.message });
        return;
    }
}

```

```
// Error de no encontrado (Not Found)
if (error.message.includes('no encontrado') ||
    error.message.includes('no existe')) {
    res.status(404).json({ error: error.message });
    return;
}

// Error genérico (Internal Server Error)
res.status(500).json({
    error: 'Error interno del servidor'
});
}
}
```

4. Implementación de Routes

4.1 Crear la Carpeta y Archivo

```
# Crear la carpeta routes
mkdir -p src/infrastructure/routes

# Crear el archivo de rutas
touch src/infrastructure/routes/employee.routes.ts
```

4.2 Código de Routes

```
// 📁 src/infrastructure/routes/employee.routes.ts

import { Router } from 'express';
import { EmployeeController } from '../controllers/employee.controller.js';

/**
 * EmployeeRoutes
 * =====
 * Configura las rutas HTTP para el sistema de empleados.
 *
 * @example
 * ```typescript
 * const router = EmployeeRoutes(employeeController);
 * app.use('/api/employees', router);
 * ```
 */
export function EmployeeRoutes(controller: EmployeeController): Router {
    const router = Router();

    // =====
    // RUTAS PRINCIPALES
    // =====

    /**
     * POST /api/employees
     * Crear un nuevo empleado
     */
    router.post('/', controller.createEmployee);

    /**
     * GET /api/employees
     * Listar todos los empleados
     */
    router.get('/', controller.getAllEmployees);

    /**
     * GET /api/employees/stats
     * Obtener estadísticas
     */
    router.get('/stats', controller.getStatistics);
}
```

```
/**
 * GET /api/employees/salary-range
 * Buscar por rango de salario
 */
router.get('/salary-range', controller.getEmployeesBySalaryRange);
```

```
/**
 * GET /api/employees/:id
 * Obtener un empleado por ID
 */
router.get('/:id', controller.getEmployeeById);
```

```
/**
 * PUT /api/employees/:id
 * Actualizar un empleado
 */
router.put('/:id', controller.updateEmployee);
```

```
/**
 * DELETE /api/employees/:id
 * Eliminar un empleado
 */
router.delete('/:id', controller.deleteEmployee);
```

```
/**
 * PATCH /api/employees/:id/salary
 * Aumentar salario
 */
router.patch('/:id/salary', controller.increaseSalary);
```

```
return router;
```

```
}
```

5. Integración con Express Server

5.1 Actualizar el Server

```
// 📁 src/infrastructure/server.ts

import express, { Application, Request, Response, NextFunction } from 'express';
import cors from 'cors';
import { EmployeeController } from '../controllers/employee.controller.js';
import { EmployeeRoutes } from '../routes/employee.routes.js';
import { EmployeeService } from '../application/services/employee.service.js';
import { InMemoryEmployeeRepository } from '../repositories/employee.inmemory.repository.js';

/**
 * Server
 * =====
 * Configura y ejecuta el servidor Express.
 */
export class Server {
  private app: Application;
  private port: number;

  constructor(port: number = 3000) {
    this.app = express();
    this.port = port;

    this.setupMiddleware();
    this.setupRoutes();
  }

  /**
   * Configura los middlewares de Express
   */
  private setupMiddleware(): void {
    // JSON parser
    this.app.use(express.json());

    // CORS
    this.app.use(cors());

    // Logging de peticiones
    this.app.use((req: Request, res: Response, next: NextFunction) => {
```



```

        console.log(`🚀 ${req.method} ${req.url}`);
        next();
    });

    // Manejo de errores global
    this.app.use((err: any, req: Request, res: Response, next: NextFunction) => {
        console.error('❌ Error global:', err.message);
        res.status(500).json({
            error: 'Error interno del servidor'
        });
    });
}

/**
 * Configura las rutas de la aplicación
 */
private setupRoutes(): void {
    // =====
    // Health Check
    // =====
    this.app.get('/health', (req: Request, res: Response) => {
        res.status(200).json({
            status: 'OK',
            timestamp: new Date().toISOString(),
            uptime: process.uptime()
        });
    });

    // =====
    // Ruta raíz
    // =====
    this.app.get('/', (req: Request, res: Response) => {
        res.status(200).json({
            message: '🚀 API de Empleados - Arquitectura Hexagonal',
            version: '1.0.0',
            endpoints: {
                'GET /health': 'Health check',
                'GET /api/employees': 'Listar empleados',
                'GET /api/employees/stats': 'Estadísticas',
                'GET /api/employees/salary-range?min=X&max=Y': 'Buscar por rango',
                'GET /api/employees/:id': 'Obtener empleado',
                'POST /api/employees': 'Crear empleado',
                'PUT /api/employees/:id': 'Actualizar empleado',
            }
        });
    });
}

```

```

        'PATCH /api/employees/:id/salary': 'Aumentar salario',
        'DELETE /api/employees/:id': 'Eliminar empleado'
    }
  });
});

// =====
// Configurar dependencias (Composition Root)
// =====
const employeeRepository = new InMemoryEmployeeRepository();
const employeeService = new EmployeeService(employeeRepository);
const employeeController = new EmployeeController(employeeService);

// =====
// Registrar rutas
// =====
this.app.use('/api/employees', EmployeeRoutes(employeeController));

// 404 - Ruta no encontrada
this.app.use((req: Request, res: Response) => {
  res.status(404).json({
    error: `Ruta no encontrada: ${req.method} ${req.url}`
  });
});
}

/**
 * Inicia el servidor
 */
start(): void {
  this.app.listen(this.port, () => {
    console.log(`🚀 Servidor ejecutándose en http://localhost:${this.port}`);
    console.log(`📊 Health check: http://localhost:${this.port}/health`);
    console.log(`📄 API: http://localhost:${this.port}/api/employees`);
  });
}

/**
 * Obtiene la instancia de Express (para pruebas)
 */
getApp(): Application {
  return this.app;
}

```

```
}  
}
```

5.2 Actualizar app.ts

```
// 📁 src/app.ts
```

```
import { Server } from './infrastructure/server.js';
```

```
/**
```

```
 * Punto de entrada principal de la aplicación
```

```
 */
```

```
const server = new Server(3000);
```

```
server.start();
```

6. Manejo de Errores

6.1 Middleware de Errores Global

```
// 📁 src/infrastructure/middlewares/error.middleware.ts

import { Request, Response, NextFunction } from 'express';

/**
 * Tipos de errores personalizados
 */
export class AppError extends Error {
  public readonly statusCode: number;
  public readonly isOperational: boolean;

  constructor(message: string, statusCode: number, isOperational: boolean = true) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = isOperational;
    Error.captureStackTrace(this, this.constructor);
  }
}

export class ValidationError extends AppError {
  constructor(message: string) {
    super(message, 400);
  }
}

export class NotFoundError extends AppError {
  constructor(message: string) {
    super(message, 404);
  }
}

export class ConflictError extends AppError {
  constructor(message: string) {
    super(message, 409);
  }
}

/**
```

```

* Middleware de manejo de errores global
*/
export function errorHandler(
  err: any,
  req: Request,
  res: Response,
  next: NextFunction
): void {
  console.error('❌ Error:', err);

  // Errores personalizados
  if (err instanceof AppError) {
    res.status(err.statusCode).json({
      error: err.message
    });
    return;
  }

  // Errores de validación (Joi, class-validator, etc.)
  if (err.name === 'ValidationError') {
    res.status(400).json({
      error: 'Error de validación',
      details: err.details
    });
    return;
  }

  // Errores de base de datos
  if (err.code === '23505') { // PostgreSQL unique violation
    res.status(409).json({
      error: 'El recurso ya existe'
    });
    return;
  }

  // Error genérico
  res.status(500).json({
    error: 'Error interno del servidor'
  });
}

```

7. Pruebas de la API

7.1 Archivo de Pruebas

```
// 📁 src/test-api.ts

import { Server } from '../infrastructure/server.js';

async function testAPI() {
  console.log('🔧 PROBANDO API DE EMPLEADOS');
  console.log('=====\\n');

  const server = new Server(3000);
  const app = server.getApp();

  // Usar supertest para pruebas HTTP
  const request = (await import('supertest')).default;

  console.log('1. Health Check');
  const healthResponse = await request(app)
    .get('/health')
    .expect(200);
  console.log(`✅ ${healthResponse.body.status}\\n`);

  console.log('2. Crear empleado');
  const createResponse = await request(app)
    .post('/api/employees')
    .send({
      cedula: '1234567890',
      nombres: 'Ana García',
      salario: 1500
    })
    .expect(201);

  const employeeId = createResponse.body.data.id;
  console.log(`✅ Creado: ${createResponse.body.data.nombres} (${employeeId})\\n`);

  console.log('3. Obtener empleado');
  const getResponse = await request(app)
    .get(`/api/employees/${employeeId}`)
    .expect(200);
  console.log(`✅ Encontrado: ${getResponse.body.data.nombres}\\n`);
```

```

console.log('4. Listar empleados');
const listResponse = await request(app)
  .get('/api/employees')
  .expect(200);
console.log(`    ✅ Total: ${listResponse.body.count} empleados\n`);

console.log('5. Actualizar empleado');
const updateResponse = await request(app)
  .put(`/api/employees/${employeeId}`)
  .send({
    nombres: 'Ana María García',
    salario: 1800
  })
  .expect(200);
console.log(`    ✅ Actualizado: ${updateResponse.body.data.nombres}\n`);

console.log('6. Aumentar salario');
await request(app)
  .patch(`/api/employees/${employeeId}/salary`)
  .send({ percentage: 10 })
  .expect(200);
console.log(`    ✅ Salario aumentado\n`);

console.log('7. Estadísticas');
const statsResponse = await request(app)
  .get('/api/employees/stats')
  .expect(200);
console.log(`    ✅ Total: ${statsResponse.body.data.total}`);
console.log(`    ✅ Promedio: ${statsResponse.body.data.averageSalary}\n`);

console.log('8. Eliminar empleado');
await request(app)
  .delete(`/api/employees/${employeeId}`)
  .expect(200);
console.log(`    ✅ Empleado eliminado\n`);

console.log('🎉 TODAS LAS PRUEBAS DE LA API PASARON!');
}

testAPI().catch(console.error);

```

7.2 Actualizar Scripts

```
{
  "scripts": {
    "dev": "tsx src/app.ts",
    "build": "tsc",
    "start": "node dist/app.js",
    "test:service": "tsx src/test-service.ts",
    "test:api": "tsx src/test-api.ts",
    "test:all": "npm run test:service && npm run test:api"
  }
}
```

8. Ejercicios de Profundización

Ejercicio 1: Agregar Endpoint de Búsqueda

```
// 🛠 TAREA: Agrega un endpoint para buscar empleados por nombre
// GET /api/employees/search?query=texto

// 1. Agrega el método en EmployeeController
// 2. Agrega la ruta en EmployeeRoutes
// 3. Agrega el método en EmployeeService
```

Ejercicio 2: Paginación

```
// 🛠 TAREA: Implementa paginación en GET /api/employees
// GET /api/employees?page=1&limit=10

// 1. Modifica el método getAllEmployees para aceptar parámetros de paginación
// 2. Agrega metadatos de paginación en la respuesta
```


Ejercicio 3: Middleware de Validación

```
// 🛠 TAREA: Crea un middleware de validación para DTOs

// 📁 src/infrastructure/middlewares/validation.middleware.ts

export function validateRequest(schema: any) {
  return (req: Request, res: Response, next: NextFunction) => {
    // Validar req.body contra el schema
    // Si es válido, next()
    // Si no, res.status(400).json({ errors })
  };
}
```

9. Checklist de Completitud

Para el Estudiante:

- ☐ Entendí el rol del Controller como Adaptador de Entrada
- ☐ Implementé EmployeeController en infrastructure/controllers/
- ☐ Implementé EmployeeRoutes en infrastructure/routes/
- ☐ Integré las rutas con el servidor Express
- ☐ Creé el endpoint POST /api/employees para crear
- ☐ Creé el endpoint GET /api/employees para listar
- ☐ Creé el endpoint GET /api/employees/:id para obtener
- ☐ Creé el endpoint PUT /api/employees/:id para actualizar
- ☐ Creé el endpoint DELETE /api/employees/:id para eliminar
- ☐ Creé el endpoint PATCH /api/employees/:id/salary para aumentar salario
- ☐ Creé el endpoint GET /api/employees/stats para estadísticas
- ☐ Creé el endpoint GET /api/employees/salary-range para búsqueda por rango
- ☐ Implementé manejo de errores HTTP (400, 404, 409, 500)
- ☐ Probé todos los endpoints con Postman/curl
- ☐ Ejecuté npm run test:api exitosamente

Notas Generales:

- ☐ Controller es delgado (solo maneja HTTP)

- ☐ No hay lógica de negocio en el Controller
- ☐ Los errores se manejan apropiadamente
- ☐ Las rutas están organizadas
- ☐ La inyección de dependencias funciona correctamente
- ☐ La API responde con los códigos HTTP correctos
- ☐ Las pruebas cubren todos los casos de uso

Resumen de lo Aprendido





Verificación Final

1. Iniciar el servidor

```
npm run dev
```

2. Probar con curl (en otra terminal)

```
curl http://localhost:3000/health
```

3. Crear un empleado

```
curl -X POST http://localhost:3000/api/employees \  
  -H "Content-Type: application/json" \  
  -d '{"cedula":"1234567890","nombres":"Ana García","salario":1500}'
```

4. Listar empleados

```
curl http://localhost:3000/api/employees
```

5. Obtener estadísticas

```
curl http://localhost:3000/api/employees/stats
```